PHASE 6 — API, Frontend, Docker, and Final Delivery

Objective

Integrate the completed research agent into a usable application and prepare it for evaluation.

Do not rewrite the core agent architecture unless a genuine integration problem requires it.

Step 1 — FastAPI

Implement the required endpoints.

GET /health

Return HTTP 200 when the application is healthy.

The health endpoint should not unnecessarily depend on an external LLM API call.

POST /ask

Accept:

```
{
"question": "What are the contraindications of warfarin?"
}
```

Validate:

- Required question.
- Non-empty input.
- Reasonable input length.

Return a structured response containing the answer, citations, and relevant status information.

Use Pydantic request and response schemas.

Handle exceptions and return appropriate HTTP status codes.

Step 2 — Application lifecycle

Initialize the document corpus, embeddings, and in-memory index appropriately.

Avoid generating document embeddings on every user request.

Prefer startup initialization or another clearly justified lifecycle strategy.

Handle startup failures explicitly.

Step 3 — React frontend

Build a minimal React + Vite interface.

The user should be able to:

- Enter a pharmacology question.
- Submit it.
- See a loading state.
- Read the generated answer.
- Inspect source citations.
- See an insufficient-evidence response when appropriate.
- Receive useful error messages.

Keep the interface clean and functional.

Do not spend excessive time on visual effects.

Step 4 — Docker

Review and complete the Dockerfile.

The required commands must work:

docker build -t agent .

docker run -p 8000:8000 --env-file .env agent

Ensure:

- Correct Python runtime.
- Dependencies installed reproducibly.
- Application starts correctly.
- Required environment variables are read.
- Frontend production build is available if serving through FastAPI.
- No secrets are copied into the image.
- The application can access the bundled corpus.

Use appropriate security practices, including a non-root runtime user where feasible.

Step 5 — Testing

Run:

- Unit tests.
- API tests.
- Integration tests.
- Docker build.
- Container startup.
- Health endpoint test.
- Ask endpoint test.

Verify all four mandatory evaluation questions.

Do not hardcode expected answers.

Step 6 — README

Document:

- Project purpose.
- Architecture.
- Technology stack.
- Project structure.
- Setup instructions.
- Environment variables.
- Running locally.
- Running with Docker.
- API examples.
- Testing instructions.
- Known limitations.
- Medical disclaimer.

Step 7 — Final presentation

Prepare a technical explanation suitable for a 10-minute presentation.

Cover:

1. Problem and objective.
2. Architecture.
3. Document ingestion and chunking.
4. Embeddings and semantic retrieval.
5. RAG.
6. Agent planning and dynamic tools.
7. Critic and evidence verification.
8. API and frontend.

9. Docker deployment.
10. Demonstration and limitations.

Final acceptance criteria

The project is complete only when:

- The application builds successfully.
- The container starts with the required command.
- GET /health returns HTTP 200.
- POST /ask accepts a valid question.
- Answers include grounded citations.
- The agent dynamically selects local tools.
- The critic reviews the answer.
- Insufficient evidence is handled safely.
- The four mandatory evaluation scenarios are tested.
- The README explains how to run the application.

Constraints

- Do not claim tests passed unless they were executed.
- Do not commit automatically.
- Do not expose secrets.
- Do not leave unfinished placeholder implementations.
- Report all known limitations honestly.

Deliverables

Provide a final engineering report including:

- Architecture summary.
- Implemented functionality.
- Test results.
- Docker verification.
- Evaluation results.
- Known limitations.
- Final repository status.